

Platform Foundation

What we stand up first — foundation pieces and repository layout every later module depends on.

Purpose. Stand up the core platform every later module depends on: a durable store for companies, users, devices, policies, and activity; a secure front door for console and agent traffic; a repeatable local staging environment; health checks for demos; rate limits so one noisy source cannot starve others; and a simple Linux install path so managed PCs can join. Without this foundation, signup, the admin console, and the endpoint agent cannot function.

1. What “foundation” means here

FOUNDATION PIECE	WHY IT EXISTS
Project structure	Known folders for backend, frontend, agent, deploy, scripts — so work lands in the right place
Local staging	One Compose stack the team can start for demos and testing
Company data model	Organizations and related tables every feature hangs off
Customer isolation	One company never sees another company’s data
Health & readiness	Operators know the stack is alive before a walkthrough
Secure gateway	Front door + traffic limits for console and agents
Endpoint install	Sealed Linux package so protection can start on a PC

2. Project structure (repository)

Top-level layout of the AI-SPM monorepo. This is the map for foundation work — not the tech catalog (that lives in Application Architecture).

```
AI-SPM/
├── backend/                # Security core (Python FastAPI package ai_spm)
│   ├── src/ai_spm/        # Application code
│   ├── alembic/           # Database migrations
│   ├── tests/             # Unit, integration, security tests
│   └── scripts/           # Seed helpers (dev tenant, platform admin)
├── frontend/              # Admin + platform React SPA
├── agent/                 # Rust endpoint agent workspace
├── deploy/                # Docker Compose, Kong, K8s, Prometheus, Grafana, systemd
├── scripts/               # install-agent, mitmproxy web UI, cert helpers
├── docs/                 # Setup, demos, foundation / architecture docs
├── infrastructure/        # Dev / local CA material
├── Makefile               # up / migrate / seed / test / agent-build
└── AGENTS.md              # Coding-agent source of truth
```

FOLDER	BELONGS HERE	DOES NOT BELONG HERE
backend/	Domain, HTTP surfaces, services, adapters, migrations, tests	React assets, Rust agent binaries
frontend/	Console UI pages, hooks, stores, client types	Server secrets, pipeline business rules
agent/	Endpoint daemon, MITM, enrollment client, installer crates	Gateway business logic (keep in backend services)
deploy/	Compose, Kong config, K8s, monitoring, systemd templates	Application feature code
scripts/	Linux install engine, mitmproxy addon, cert generation	Runtime product screens